

Progetto Di Reti Logiche

A.A. 2025–2026

Relazione

Studenti:

10893716

10912026

Docente di riferimento:

Fabio Salice

Indice

1	Introduzione	2
2	Architettura	2
2.1	Porte di Interfaccia	2
2.2	Segnali Interni — Registri (Flip-Flop)	2
2.3	Sensitivity List del Processo Combinatorio	3
2.4	Moduli e processi	3
2.4.1	Processo <code>state_reg</code>	3
2.4.2	Processo <code>datapath_regs</code>	3
2.4.3	Processo <code>comb</code>	3
2.5	Diagramma degli Stati — Riepilogo	4
3	Scelte progettuali	4
3.1	Gestione della RAM sincrona write-first	4
3.2	Controllo della lista vuota in <code>S_DECODE</code>	4
4	Risultati sperimentali	4
4.1	Sintesi	4
4.2	Simulazioni	4
5	Conclusioni	4

1 Introduzione

L'obiettivo del progetto è realizzare in VHDL un componente che gestisce una lista ordinata di task memorizzata su una RAM esterna.

Ogni task è codificato su 8 bit con i primi 6 bit più significativi che rappresentano ID_TASK (6 bit) e gli ultimi 2 bit che rappresentano PRIORITY (2 bit), dove la priorità 0 è la massima e 3 la minima. La RAM è organizzata con l'indirizzo 0 che contiene il numero di task presenti (num_tasks), mentre gli indirizzi 1..N contengono i task ordinati per priorità crescente.

Il modulo implementato supporta quattro operazioni, selezionate dal segnale i_op:

- **00 – Decremento Priorità:** incrementa il valore numerico di priorità di tutti i task con limite a 3 (priorità minima).
- **01 – Estrazione Primo Task:** rimuove il task in testa alla lista, shifta gli altri verso sinistra e restituisce l'ID estratto.
- **10 – Inserimento Task:** inserisce un nuovo task mantenendo l'ordinamento per priorità, mettendolo in coda rispetto a task di pari priorità.
- **11 – Svuota Lista:** azzera il contatore in indirizzo 0 rendendo la lista vuota.

Il componente si interfaccia con la logica esterna attraverso un protocollo di handshake i_start/o_done e con una RAM sincrona di tipo write-first fornita nel testbench.

2 Architettura

L'architettura è organizzata come una macchina a stati finiti (FSM) con datapath.

2.1 Porte di Interfaccia

La Tabella 1 riassume i segnali di I/O del componente.

Segnale	Dir.	Bit	Descrizione
i_clk	IN	1	Clock di sistema (campionamento sul fronte di SALITA).
i_rst	IN	1	Reset ASINCRONO attivo alto. Forza la FSM in S_RESET.
i_start	IN	1	Handshake: portato a 1 per avviare un'operazione.
i_task_id	IN	6	ID del task da inserire (usato da OP10). Valori 1..63.
i_task_priority	IN	2	Priorità del task da inserire (00=max, 11=min).
i_op	IN	2	Selezione operazione: 00/01/10/11.
o_done	OUT	1	Handshake: uscita COMBINATORIA da S_DONE.
o_task_id	OUT	6	ID task estratto (OP01). 0 se lista vuota.
o_mem_addr	OUT	16	Indirizzo RAM (range 0..num_tasks).
i_mem_data	IN	8	Dato letto dalla RAM (valido dopo 1 ciclo).
o_mem_data	OUT	8	Dato da scrivere in RAM.
o_mem_we	OUT	1	Write Enable: 1=scrittura, 0=lettura.
o_mem_en	OUT	1	Enable memoria: 1 per ogni accesso.

Tabella 1: Segnali di interfaccia del modulo.

2.2 Segnali Interni — Registri (Flip-Flop)

Ogni segnale interno ha una versione **current_*** (valore attuale) e **next_*** (calcolato dalla logica combinatoria).

Segnale	Bit	Ruolo e motivazione della scelta
<code>current_state</code>	enum	Stato corrente della FSM. Registro centrale di controllo.
<code>num_tasks</code>	8	Copia locale di RAM[0]. Evita riletture continue della RAM.
<code>current_addr</code>	16	Indirizzo corrente per scansioni (OP00, OP01 shift, OP10).
<code>target_addr</code>	16	Posizione di inserimento (OP10). Salvata per l'uso post-shift.
<code>extracted_id</code>	6	ID estratto (OP01). Salvato perché <code>i_mem_data</code> viene sovrascritto.

Tabella 2: Registri interni del datapath.

2.3 Sensitivity List del Processo Combinatorio

La sensitivity list deve contenere TUTTI i segnali letti nel processo per evitare discrepanze tra simulazione e sintesi.

Segnale	Perché è necessario nella sensitivity list
<code>current_state</code>	È il selettore del case: ogni cambio stato deve rieseguire la logica.
<code>i_start</code> , <code>i_op</code>	Ingressi primari letti in <code>S_IDLE</code> e <code>S_DECODE</code> .
<code>i_task_id</code> , ...	Usati per costruire il byte da scrivere o confrontare priorità.
Registri interni	Condizioni di terminazione, indirizzi e confronti.
<code>i_mem_data</code>	Dato letto dalla RAM, usato in quasi tutte le fasi operative.

Tabella 3: Dettaglio della Sensitivity List del processo `comb`.

2.4 Moduli e processi

2.4.1 Processo `state_reg`

Implementa il registro di stato con reset asincrono attivo alto: se `i_rst='1'` la FSM viene forzata in `S_RESET`. Sul fronte di salita del clock, `current_state` assume il valore di `next_state`.

2.4.2 Processo `datapath_regs`

Aggiorna i registri interni (`num_tasks`, `current_addr`, ecc.) sul fronte di salita del clock.

2.4.3 Processo `comb`

Processo combinatorio che determina le uscite e lo stato successivo. Include una sensitivity list completa come dettagliato nella Tabella 3.

2.5 Diagramma degli Stati — Riepilogo

La Tabella 4 riassume le transizioni principali della FSM.

Stato	Condizione di uscita	Stato successivo
S_RESET	-	S_IDLE
S_IDLE	<code>i_start = '1'</code>	S_LOAD_COUNT
S_LOAD_COUNT	-	S_DECODE
S_DECODE	<code>i_op / lista vuota</code>	S_OP00_READ / S_OP01_READ_FIRST / ...
S_OP00_READ	-	S_OP00_MODIFY
S_OP00_MODIFY	<code>curr_addr < num_tasks</code>	S_OP00_READ
S_OP00_MODIFY	<code>curr_addr = num_tasks</code>	S_WAIT_DONE
S_OP01_READ_FIRST	-	S_OP01_SAVE
S_OP01_SAVE	<code>num_tasks = 1</code>	S_OP01_UPDATE
S_OP01_SAVE	<code>num_tasks > 1</code>	S_OP01_SHIFT_READ
S_OP01_SHIFT_READ	-	S_OP01_SHIFT_WRITE
S_OP01_SHIFT_WRITE	<code>current_addr = num_tasks</code>	S_OP01_UPDATE
S_OP01_SHIFT_WRITE	<code>current_addr < num_tasks</code>	S_OP01_SHIFT_READ
S_OP01_UPDATE	-	S_WAIT_DONE
S_OP10_FIND_READ	-	S_OP10_COMPARE
S_OP10_COMPARE	<code>i_mem_data(1 downto 0) > i_task_priority</code>	S_OP10_SHIFT_READ
S_OP10_COMPARE	<code>curr_addr >= num_tasks</code>	S_OP10_INSERT
S_OP10_COMPARE	else	S_OP10_FIND_READ
S_OP10_SHIFT_READ	-	S_OP10_SHIFT_WRITE
S_OP10_SHIFT_WRITE	<code>current_addr = target_addr</code>	S_OP10_INSERT
S_OP10_SHIFT_WRITE	<code>current_addr > target_addr</code>	S_OP10_SHIFT_READ
S_OP10_INSERT	-	S_OP10_UPDATE_SIZE
S_OP10_UPDATE_SIZE	-	S_WAIT_DONE
S_OP11_CLEAR	-	S_WAIT_DONE
S_WAIT_DONE	-	S_DONE
S_DONE	<code>i_start = '0'</code>	S_IDLE

Tabella 4: Tabella delle transizioni della FSM.

3 Scelte progettuali

3.1 Gestione della RAM sincrona write-first

La RAM è sincrona con una latenza di un ciclo in lettura. Utilizziamo un pattern di accesso a due cicli (lettura/elaborazione) sfruttando il fatto che il dato è stabile nel ciclo $T+1$.

3.2 Controllo della lista vuota in S_DECODE

Lo stato S_DECODE verifica se la lista è vuota per diramare il flusso verso stati speciali, risparmiando cicli di clock (dire magari che prima si era scelto di usare `checkempty` e poi è cambiata l'idea siccome erano inutili).

4 Risultati sperimentali

4.1 Sintesi

4.2 Simulazioni

5 Conclusioni